Cherry River Integration Project - Complete Summary & Handoff Document

Last Updated: December 29, 2025
Project Status: Phase 1 Complete (Odoo → Relevance) | Phase 2 In Progress (SAQ Integration)
Purpose: This document provides complete context for continuing SAQ integration work

---

📋 TABLE OF CONTENTS

---

1. PROJECT OVERVIEW

Business Context

Cherry River is a beverage manufacturing company that produces RTD (Ready-to-Drink) cocktails, mocktails, and alcoholic beverages. They sell through multiple channels, with **SAQ (Société des alcools du Québec)** being their primary sales channel.

Core Business Challenge

Cherry River needed a unified system to:

- Track inventory across manufacturing, warehouse, and sales
- Forecast demand based on actual sales data
- Optimize production planning based on real consumption
- Manage procurement intelligently

- Prevent stockouts of popular products sold through SAQ

Project Goal

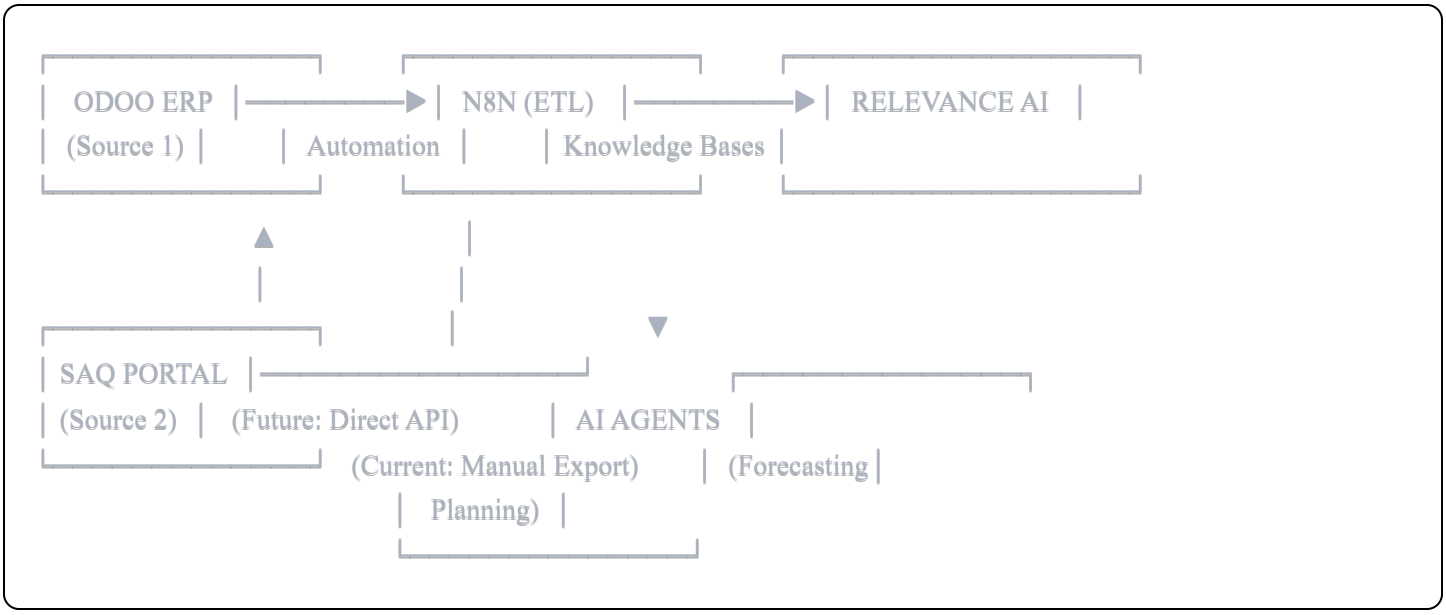
Create an AI-powered inventory forecasting and production planning system by:

1. ✅ **COMPLETE**: Syncing Odoo ERP data (inventory, BOMs, purchases, production) to Relevance AI Knowledge Bases
2. 🔄 **IN PROGRESS**: Integrating SAQ sales data to provide real-time demand forecasting
3. ⏳ **PENDING**: Building AI agents that use combined data for intelligent recommendations

---

2. SYSTEM ARCHITECTURE

Data Flow Overview



Technology Stack

Component	Technology	Purpose
ERP System	Odoo 16 (Cloud)	Master data source for products, inventory, BOMs, purchases
Automation Platform	N8N (Self-hosted)	ETL processes, data transformation, scheduled syncs
AI/Knowledge Platform	Relevance AI	Knowledge bases, AI agents, query interface
Sales Data Source	SAQ Portal	Actual sales transactions (B2B customer portal)

Why This Architecture?

Previous Attempt: Make.com

- ❌ **Failed**: 45-minute timeout limit
- ❌ **Too Slow**: 5 seconds per record = 21 hours for 15,000 records
- ❌ **Not Scalable**: No bulk operations

Current Solution: N8N

- ✅ **Fast**: Batch processing (60 records/batch)
- ✅ **Reliable**: No timeout limits
- ✅ **Flexible**: Custom JavaScript transformations
- ✅ **Result**: ~4 hours for 15,000 records (vs 21 hours with Make)

---

3. CURRENT IMPLEMENTATION STATUS

✅ PHASE 1: ODOO INTEGRATION (COMPLETE)

Successfully implemented N8N workflows syncing the following Odoo entities to Relevance AI:

Odoo Model	KB Name	Records	Status	Key Fields
product.product	product_data_odoo	200	✅ Live	id, name, default_code, list_price, standard_price, categ_id
stock.quant	stock_quant_odoo	200	✅ Live	id, product_id, location_id, quantity, reserved_quantity, available_quantity
purchase.order	purchase_data_odoo	110	✅ Live	id, name, partner_id, date_order, amount_total, state
purchase.order.line	purchase_order_line_odoo	320	✅ Live	id, order_id, product_id, product_qty, price_unit, date_planned
mrp.bom	manufacturing_data_odoo	190	✅ Live	id, product_id, code, type, product_qty
mrp.bom.line	mrp_bom_line	1,893	✅ Live	id, bom_id, product_id, product_qty (ingredients/components)
mrp.production	mrp_production	2	✅ Live	id, name, product_id, product_qty, bom_id, state, dates
stock.move	stock_movement_history	305	✅ Live	id, date, product_id, quantity, location_id, location_dest_id, origin
sale.order	sale_order	29	✅ Live	id, name, partner_id, date_order, amount_total, state
sale.order.line	sale_order_line	97	✅ Live	id, order_id, product_id, product_uom_qty, price_unit

Key Achievement: All Odoo operational data now flows automatically into Relevance AI every 15 minutes.

🔄 PHASE 2: SAQ INTEGRATION (IN PROGRESS)

Current Status: Initial exploration and data structure analysis completed

What is SAQ?

- **SAQ** = Société des alcools du Québec (Quebec Alcohol Corporation)
- Government-owned corporation with monopoly on alcohol distribution in Quebec
- Cherry River's **primary B2B customer** for alcoholic beverage sales

- SAQ provides a B2B portal where suppliers (like Cherry River) can view their sales data

SAQ Data Importance:

- Contains **actual point-of-sale transactions** from SAQ stores across Quebec
- Represents ~80% of Cherry River's total sales volume
- Critical for accurate demand forecasting
- Current Odoo sales data (29 orders) represents only internal/direct sales, NOT SAQ sales

SAQ Portal Access:

- Cherry River has login credentials to SAQ B2B supplier portal
- Portal provides sales reports, inventory levels at SAQ warehouses, and transaction history
- Currently: Data extracted manually via Excel/CSV exports
- Goal: Automate data extraction and sync to Relevance AI

---

4. KNOWLEDGE BASE STRUCTURE

Complete Knowledge Base Schema

Each Knowledge Base in Relevance AI contains structured data with the following schemas:

`product_data_odoo` (200 documents)

Purpose: Master product catalog

```
Fields:
- id (int) - Odoo product ID
- name (string) - Product name
- default_code (string) - SKU/product code
- list_price (float) - Selling price
- standard_price (float) - Cost price
- categ_id (int) - Category ID
- categ_name (string) - Category name
```

`stock_quant_odoo` (200 documents)

Purpose: Real-time inventory levels by location

Fields:
- id (int) - Stock quant ID
- product_id (int) - Product ID
- product_name (string) - Product name
- location_id (int) - Warehouse location ID
- location_name (string) - Location name
- quantity (float) - Total quantity
- reserved_quantity (float) - Quantity reserved for orders
- available_quantity (float) - Calculated: quantity - reserved_quantity
- in_date (datetime) - Date stock entered location
- lot_id (int, optional) - Lot/batch ID
- lot_name (string, optional) - Lot name

`stock_movement_history` (305 documents)

Purpose: Historical inventory movements (production, transfers, sales)

Fields:
- id (int) - Move ID
- date (datetime) - Movement date
- product_id (int) - Product ID
- product_name (string) - Product name
- product_qty (float) - Quantity moved
- quantity (float) - Quantity moved (duplicate field)
- product_uom (int) - Unit of measure ID
- product_uom_name (string) - Unit name (kg, L, units)
- location_id (int) - Source location ID
- location_from (string) - Source location name
- location_dest_id (int) - Destination location ID
- location_to (string) - Destination location name
- state (string) - Movement state (done, cancel)
- origin (string) - Source document reference
- reference (string) - Movement reference
- picking_type_id (int) - Operation type ID
- picking_type_name (string) - Operation type name
- price_unit (float) - Unit price
- year (int) - Calculated from date
- month (int) - Calculated from date
- week (int) - Calculated from date
- create_date (datetime) - Record creation date
- write_date (datetime) - Last update date

purchase_data_odoo **(110 documents)**

Purpose: Purchase order headers

Fields:
- id (int) - PO ID
- name (string) - PO number (e.g., P00108)
- partner_id (int) - Supplier ID
- partner_name (string) - Supplier name
- date_order (datetime) - Order date
- date_planned (datetime) - Expected delivery date
- date_approve (datetime) - Approval date
- amount_total (float) - Total with taxes
- amount_untaxed (float) - Total before taxes
- currency_name (string) - Currency (CAD)
- state (string) - PO state (purchase, done)

purchase_order_line_odoo **(320 documents)**

Purpose: Purchase order line items

Fields:
- id (int) - Line ID
- order_id (int) - Parent PO ID
- order_name (string) - Parent PO number
- product_id (int) - Product ID
- product_name (string) - Product name
- product_qty (float) - Ordered quantity
- product_uom (int) - Unit of measure ID
- product_uom_name (string) - Unit name
- price_unit (float) - Unit price
- price_subtotal (float) - Line total before taxes
- price_total (float) - Line total with taxes
- date_planned (datetime) - Expected delivery date
- date_order (datetime) - Order date (from parent)
- description (string) - Line description
- state (string) - Line state

manufacturing_data_odoo **(190 documents)**

Purpose: Bill of Materials (BOM) headers

Fields:
- id (int) - BOM ID
- product_id (int) - Finished product ID
- product_name (string) - Finished product name
- product_tmpl_id (int) - Product template ID
- product_tmpl_name (string) - Product template name
- product_qty (float) - Quantity produced per BOM
- product_uom_id (int) - Unit of measure ID
- product_uom_name (string) - Unit name
- code (string) - BOM reference code
- type (string) - BOM type (normal, phantom)
- ready_to_produce (string) - Production trigger setting
- consumption (string) - Component consumption method
- create_date (datetime) - BOM creation date
- write_date (datetime) - Last update date

`mrp_bom_line` (1,893 documents) ☆ MOST IMPORTANT FOR PRODUCTION

Purpose: Detailed BOM ingredients/components for each product

Fields:
- id (int) - BOM line ID
- bom_id (int) - Parent BOM ID
- bom_name (string) - Parent BOM name
- product_id (int) - Ingredient/component product ID
- product_name (string) - Ingredient name
- product_qty (float) - Quantity needed per batch
- product_uom_id (int) - Unit of measure ID
- product_uom_name (string) - Unit name (kg, L, g)
- sequence (int) - Line order in BOM
- create_date (datetime) - Line creation date
- write_date (datetime) - Last update date

Example BOM Line:

```
json
```

```
{
  "id": 70,
  "bom_id": 11,
  "bom_name": "Assemblage RTD Margarita",
  "product_id": 31,
  "product_name": "Sel",
  "product_qty": 0.2,
  "product_uom_name": "g",
  "sequence": 1
}
```


Translation: To make one batch of RTD Margarita, you need 0.2g of Sel (Salt)

mrp_production (2 documents)

Purpose: Manufacturing orders (production history)

```
Fields:
- id (int) - Production order ID
- name (string) - Production order number
- product_id (int) - Product being manufactured
- product_name (string) - Product name
- product_qty (float) - Quantity to produce
- product_uom_id (int) - Unit of measure ID
- product_uom_name (string) - Unit name
- bom_id (int) - BOM used for production
- bom_name (string) - BOM name
- state (string) - Production state (confirmed, progress, done)
- date_planned_start (datetime) - Planned start date
- date_planned_finished (datetime) - Planned end date
- date_start (datetime) - Actual start date
- date_finished (datetime) - Actual completion date
- origin (string) - Source document
- user_id (int) - Responsible user ID
- user_name (string) - Responsible user name
- create_date (datetime) - Order creation date
- write_date (datetime) - Last update date
```

sale_order (29 documents) ⚠ LIMITED DATA

Purpose: Sales order headers (internal/direct sales only, NOT SAQ)

Fields:
- id (int) - Sales order ID
- name (string) - Sales order number (e.g., S00060)
- partner_id (int) - Customer ID
- partner_name (string) - Customer name
- date_order (datetime) - Order date
- commitment_date (datetime, optional) - Committed delivery date
- amount_total (float) - Total with taxes
- amount_untaxed (float) - Total before taxes
- currency_name (string) - Currency
- state (string) - Order state (sale, done)
- note (string, optional) - Order notes
- create_date (datetime) - Order creation date
- write_date (datetime) - Last update date

sale_order_line **(97 documents)** ⚠ **LIMITED DATA**

Purpose: Sales order line items

Fields:
- id (int) - Line ID
- order_id (int) - Parent sales order ID
- order_name (string) - Parent order number
- product_id (int) - Product ID
- product_name (string) - Product name
- product_uom_qty (float) - Ordered quantity
- product_uom (int) - Unit of measure ID
- product_uom_name (string) - Unit name
- price_unit (float) - Unit price
- price_subtotal (float) - Line total before taxes
- price_total (float) - Line total with taxes
- discount (float) - Discount percentage
- date_order (datetime) - Order date (from parent)
- description (string) - Line description
- state (string) - Line state

⚠ **CRITICAL NOTE ON SALES DATA**:

- Current Odoo sales data (29 orders, 97 lines) represents only **internal/direct sales**
- Does **NOT** include SAQ sales (which represent ~80% of total volume)
- SAQ integration is required for accurate demand forecasting

5. SAQ INTEGRATION STATUS & NEXT STEPS

5.1 Current Understanding of SAQ Data

What We Know:

1. ✅ SAQ is Cherry River's primary sales channel (~80% of volume)
2. ✅ SAQ provides a B2B supplier portal with sales data access
3. ✅ Cherry River has portal login credentials
4. ✅ Portal contains transactional sales data (store-level, date-stamped)
5. ✅ Data is currently extracted manually via Excel/CSV exports

What We Need to Determine:

1. ❓ Does SAQ provide an API for automated data extraction?
2. ❓ If no API, what is the portal's data export format? (Excel, CSV, PDF?)
3. ❓ What is the exact data structure of SAQ exports?
4. ❓ How frequently is data updated in the SAQ portal? (Daily, weekly?)
5. ❓ What is the historical data availability? (How far back can we pull?)
6. ❓ Are there any rate limits or access restrictions on data exports?

5.2 SAQ Data Requirements

Minimum Required Fields for effective forecasting:

Field	Purpose	Priority
Transaction Date	Time-series analysis, trend detection	🔴 Critical
Product ID/SKU	Match to Odoo products	🔴 Critical
Product Name	Validation, matching	🔴 Critical
Quantity Sold	Demand calculation	🔴 Critical
Store/Location	Geographic analysis	🟡 Important
Unit Price	Revenue analysis	🟡 Important
Total Sale Amount	Revenue tracking	🟡 Important
Customer/Order ID	Optional for deeper analysis	🟢 Nice-to-have

Example Expected SAQ Data Structure:

```csv
Date,Product_Code,Product_Name,Store_ID,Store_Name,Quantity,Unit_Price,Total_Amount
2025-12-15,CR-MARG-355,RTD Margarita 355ml,SAQ-1234,Montreal Downtown,48,5.75,276.00
2025-12-15,CR-MARG-355,RTD Margarita 355ml,SAQ-5678,Quebec City,24,5.75,138.00
2025-12-16,CR-AMAR-355,Mocktail Amaretto Sour,SAQ-1234,Montreal Downtown,36,4.50,162.00
```

5.3 Technical Approaches for SAQ Integration

Option A: Direct API Integration (PREFERRED)

If SAQ provides an API:

```
SAQ API ──────▶ N8N Workflow ──────▶ Relevance AI
          (scheduled, every 6-24 hours)
```

Implementation Steps:

1. Obtain SAQ API credentials and documentation
2. Create N8N workflow with HTTP Request nodes
3. Authenticate to SAQ API
4. Fetch sales transactions (filtered by date range)

5. Transform data to match Relevance AI schema
6. Batch upload to new KB: `saq_sales_data`
7. Schedule workflow to run daily

Advantages:

- ✅ Fully automated
- ✅ Real-time or near-real-time data
- ✅ Reliable and scalable
- ✅ No manual intervention needed

N8N Workflow Structure (API approach):

```
Trigger (Schedule)
  → HTTP Request (SAQ API Auth)
  → HTTP Request (Fetch Sales Data)
  → Code Node (Transform Data)
  → Split in Batches
  → HTTP Request (Relevance Tool Call)
```

Option B: Automated File Export Processing

If SAQ portal allows scheduled exports to FTP/SFTP/Email:

```
SAQ Portal ——▶ FTP/SFTP/Email ——▶ N8N ——▶ Relevance AI
    (daily export)    (monitoring)
```

Implementation Steps:

1. Configure SAQ portal to auto-export sales data daily
2. Set up FTP/SFTP server or email monitoring
3. Create N8N workflow to detect new files
4. Parse CSV/Excel files
5. Transform and upload to Relevance AI

Advantages:

- ✅ No API dependency
- ✅ Can still be automated
- ✅ Works with existing export infrastructure

N8N Workflow Structure (File monitoring approach):

```
Trigger (File Upload to FTP)
  → Read File (CSV/Excel)
  → Code Node (Parse & Transform)
  → Split in Batches
  → HTTP Request (Relevance Tool Call)
```

Option C: Manual Upload + Processing (CURRENT INTERIM)

For immediate start while investigating Options A/B:

```
SAQ Portal ──────▶ Manual Download ──────▶ Upload to N8N ──────▶ Relevance AI
          (weekly/monthly)      (HTTP endpoint)
```

Implementation Steps:

1. Create N8N webhook endpoint that accepts file uploads
2. User manually downloads SAQ export
3. User uploads file to N8N webhook URL
4. N8N processes and syncs to Relevance AI

Advantages:

- ✅ Can start immediately
- ✅ No SAQ technical requirements
- ✅ Validates data structure before automation

Disadvantages:

- ❌ Requires manual effort
- ❌ Not real-time
- ❌ Prone to delays/forgetting

5.4 Proposed SAQ Knowledge Base Structure

KB Name: `saq_sales_data`

Schema (to be confirmed with actual SAQ data):

```
Fields:
- id (string) - Unique transaction ID (generated or from SAQ)
- transaction_date (datetime) - Date of sale
- product_code (string) - Product SKU from SAQ
- product_id (int) - Matched Odoo product ID
- product_name (string) - Product name
- store_id (string) - SAQ store identifier
- store_name (string) - SAQ store name
- store_region (string) - Geographic region (Montreal, Quebec City, etc.)
- quantity_sold (float) - Units sold
- unit_price (float) - Price per unit
- total_amount (float) - Total sale amount
- year (int) - Extracted from transaction_date
- month (int) - Extracted from transaction_date
- week (int) - Extracted from transaction_date
- day_of_week (string) - Monday, Tuesday, etc.
```

Data Volume Estimate:

- If Cherry River sells ~20 SKUs through SAQ
- Across ~50-100 SAQ stores
- With daily transactions
- Expected records per month: ~30,000 - 60,000
- Annual records: ~400,000 - 700,000

5.5 Product Matching Strategy

Challenge: SAQ may use different product codes than Odoo

Solution: Create a mapping table

KB Name: `saq_product_mapping`

Schema:

Fields:
- saq_product_code (string) - SAQ's product identifier
- saq_product_name (string) - SAQ's product name
- odoo_product_id (int) - Odoo product ID
- odoo_product_name (string) - Odoo product name
- odoo_default_code (string) - Odoo SKU
- is_active (boolean) - Whether mapping is active
- notes (string) - Any mapping notes

Implementation:

1. Extract unique products from first SAQ data export
2. Manually create initial mapping file (CSV)
3. Upload mapping to Relevance AI
4. N8N workflow references mapping during transformation
5. Update mapping when new products are added

5.6 Immediate Next Steps for SAQ Integration

Priority 1: SAQ Data Discovery (URGENT)

Owner: Client (Cherry River) **Tasks**:

1. Log into SAQ B2B supplier portal
2. Navigate to sales/reports section
3. Take screenshots of:
    - Available report types
    - Sample data table/export
    - Export format options (Excel, CSV, PDF)
    - Date range options
    - Any API documentation links
4. Download sample export file (last 30 days if possible)
5. Check for "Developer" or "API" section in portal
6. Contact SAQ support to ask:
    - "Do you provide an API for suppliers to access sales data?"

- "What is the frequency of data updates?"
- "Can reports be scheduled/automated?"

Priority 2: Data Structure Analysis

Owner: Developer (You) **Tasks**:

1. Receive sample SAQ export file from client
2. Analyze column structure
3. Identify matching fields to Odoo products
4. Create product mapping template
5. Design `saq_sales_data` KB schema
6. Design `saq_product_mapping` KB schema

Priority 3: Initial Implementation

Owner: Developer **Tasks**:

1. Create N8N workflow for SAQ data processing
2. Implement data transformation logic
3. Create product matching logic
4. Set up Relevance AI KBs
5. Test with sample data
6. Document any data quality issues

Priority 4: Automation Planning

Owner: Developer + Client **Tasks**:

1. Based on SAQ capabilities, choose integration approach (A, B, or C)
2. If API exists: Obtain credentials and implement
3. If file export: Configure automation
4. If manual only: Create documentation and schedule
5. Schedule regular syncs

5.7 Success Criteria for SAQ Integration

Phase 1: Data Flowing ✅

- [ ] SAQ sales data successfully imported into Relevance AI
- [ ] At least 90 days of historical data loaded
- [ ] Product matching >95% accurate
- [ ] Data refreshed at least weekly

Phase 2: Data Quality ✅

- [ ] No duplicate transactions
- [ ] All transactions have valid dates
- [ ] All products matched to Odoo
- [ ] Data validation rules passing

Phase 3: Forecasting Ready ✅

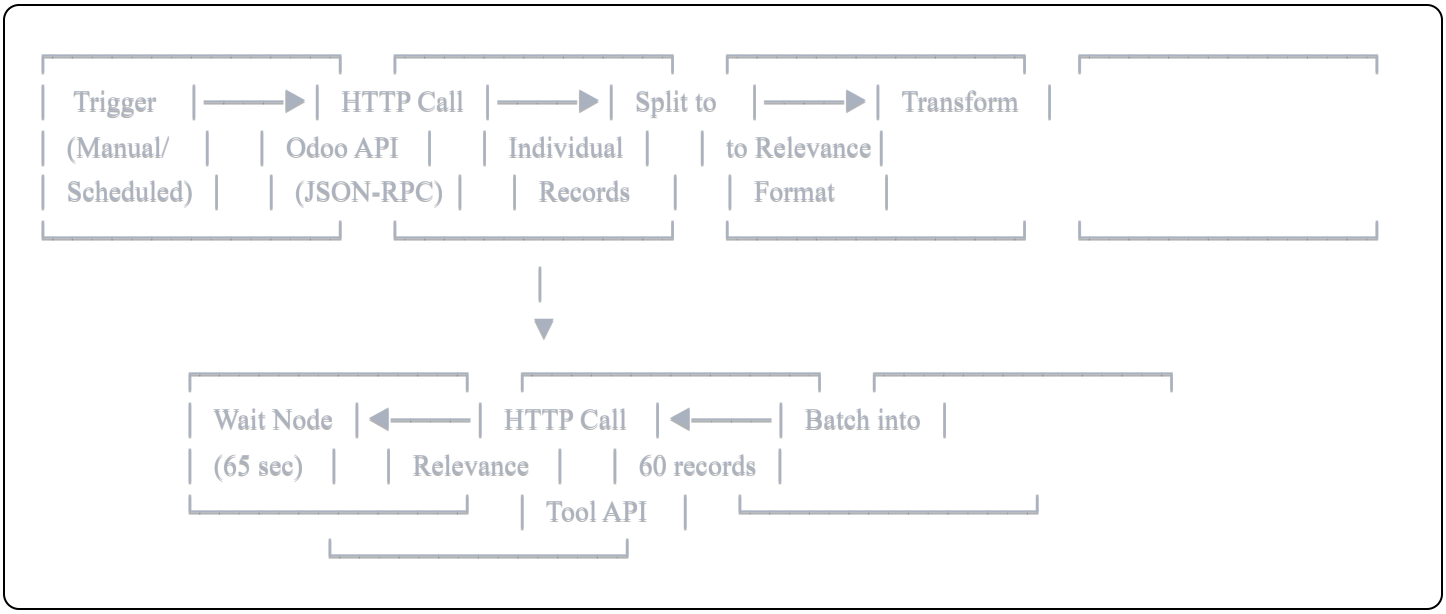
- [ ] Combined dataset (Odoo + SAQ) available to AI agents
- [ ] Time-series analysis possible (weekly, monthly trends)
- [ ] Product-level sales velocity calculable
- [ ] Geographic analysis enabled (if store data available)

---

6. TECHNICAL IMPLEMENTATION DETAILS

6.1 N8N Workflow Architecture

Core Pattern Used for All Odoo Entities:



Key Components:

1. **Trigger Node**: Manual or scheduled execution
2. **Odoo API Call**: HTTP Request to Odoo JSON-RPC endpoint
3. **Split Records**: Code node to convert API response array to individual items
4. **Transform**: Code node to map Odoo fields to Relevance schema
5. **Batch**: Split in Batches node (60 records per batch)
6. **Sync to Relevance**: HTTP Request to Relevance Tool API
7. **Wait**: 65-second delay between batches to respect rate limits

6.2 Handling Nested Relational Data (Parent-Child)

Example: Purchase Orders + Purchase Order Lines

Challenge: Each Purchase Order has multiple Purchase Order Lines (1:many relationship)

Solution: Loop through parent records, fetch children for each

```
Get POs → Split POs → Prepare PO → Send PO to Relevance
                                        ↓
                                Get PO Lines (filtered by parent order_id)
                                        ↓
                                Check if lines exist (IF node)
                                        ↓
                                Flatten & Split Lines → Prepare Line → Send Lines
                                        ↓
                                Wait → Loop back to next PO
```

Critical Code Pattern (from Node: "Make Items4"):

```javascript
// Flatten all PO responses and their lines into individual items
const allLineItems = items.flatMap(poResponse => {
  const lines = poResponse.json.result || [];

  return lines.map(line => ({
    json: {
      line: line,
      parent_order_id: line.order_id ? line.order_id[0] : null,
      parent_order_name: line.order_id ? line.order_id[1] : "",
      parent_date_order: line.date_planned || ""
    }
  }));
});

return allLineItems;
```

Why This Works:

- ✅ Handles 1 PO with 1 line
- ✅ Handles 1 PO with 100 lines
- ✅ Handles 100 POs with varying lines each
- ✅ Uses `flatMap()` to flatten nested arrays automatically

6.3 Odoo API Integration Details

Endpoint: `https://crm.cherryriver.ca/jsonrpc`

Authentication: API Key (included in request body)

Request Format (JSON-RPC):

json

```
{
  "jsonrpc": "2.0",
  "method": "call",
  "params": {
    "service": "object",
    "method": "execute_kw",
    "args": [
      "cherryriver-master-15582594",  // Database name
      33,                             // User ID
      "api_key_here",                 // API Key
      "product.product",              // Model name
      "search_read",                  // Method
      [[]],                           // Domain (filters)
      {
        "fields": ["id", "name", "default_code"],  // Fields to return
        "limit": 0                    // 0 = all records
      }
    ]
  },
  "id": 1
}
```

Response Format:

json

```
{
 "jsonrpc": "2.0",
 "id": 1,
 "result": [
  {
   "id": 15,
   "name": "Acide citrique",
   "default_code": "ING-AC-001",
   "categ_id": [8, "Ingredients"], // Many2one: [id, display_name]
   "list_price": 5.75
  },
  // ... more records
 ]
}
```

Odoo Many2one Field Format:

- Odoo returns relational fields as: `[id, "Display Name"]`
- Example: `"partner_id": [42, "Black Fly Beverages"]`
- Extract: `partner_id = [42, "Black Fly Beverages"][0]` = 42
- Extract: `partner_name = [42, "Black Fly Beverages"][1]` = "Black Fly Beverages"

6.4 Relevance AI Integration Details

Endpoint: `https://api-{region}.stack.tryrelevance.com/latest/studios/{tool_id}/trigger_webhook`

Authentication:

- Header: `Authorization: {project_id}:{api_key}`

Request Format:

```
json
```

```
{
 "knowledge_table": "product_data_odoo",
 "id_column_name": "id",
 "record_json": {
  "id": 15,
  "name": "Acide citrique",
  "list_price": 5.75,
  "standard_price": 1.00,
  "categ_id": 8,
  "categ_name": "Ingredients"
 }
}
```


Critical Rules:

1. **No Null Values**: Only include fields with valid data
2. **Conditional JSON Building**: Use JavaScript to build records dynamically
3. **One Record at a Time**: Each API call inserts one record
4. **Batching Required**: Use N8N batching to respect rate limits

Transformation Pattern (Conditional JSON):

```javascript
```

```
const transformed = items.map(item => {
  const data = item.json;

  const record = {
    id: data.id,
    name: data.name || ""
  };

  // Only add if exists
  if (data.partner_id && data.partner_id[0]) {
    record.partner_id = data.partner_id[0];
    record.partner_name = data.partner_id[1] || "";
  }

  if (data.price) record.price = data.price;

  return {
    json: {
      knowledge_table: "kb_name",
      id_column_name: "id",
      record_json: record
    }
  };
});

return transformed;
```

6.5 Batching & Rate Limiting

Settings:

- Batch Size: 60 records
- Batch Interval: 70,000 ms (70 seconds)
- Wait Between Batches: 65 seconds (additional safety)

Performance:

- 60 records per 70 seconds
- ~51 records per minute
- ~3,000 records per hour

- ~15,000 records in 5 hours

Why These Numbers:

- Relevance API has rate limits (not publicly documented)
- Testing showed 60 records/minute is safe
- 70-second interval + 65-second wait = buffer for API processing

6.6 Data Enrichment Patterns

Example: Date Enrichment for Stock Movements

```javascript
// Calculate year, month, week from date field
const moveDate = new Date(data.date);
const year = moveDate.getFullYear();
const month = moveDate.getMonth() + 1;

// Calculate week number
const startOfYear = new Date(year, 0, 1);
const pastDaysOfYear = (moveDate - startOfYear) / 86400000;
const week = Math.ceil((pastDaysOfYear + startOfYear.getDay() + 1) / 7);

record.year = year;
record.month = month;
record.week = week;
```

Example: Available Quantity Calculation

```javascript
// For stock_quant: calculate available = total - reserved
record.available_quantity = (data.quantity || 0) - (data.reserved_quantity || 0);
```

---

7. CREDENTIALS & ACCESS

7.1 Odoo ERP

- **URL**: https://crm.cherryriver.ca

- **Database**: cherryriver-master-15582594
- **User ID**: 33
- **API Key**: `c41b7126d416032f6de615f3a937df6ee48b1878`

7.2 Relevance AI

- **Region**: bcbe5a
- **Project ID**: `248e7576-e3d3-4add-ac5c-d3072d3b4567`
- **API Key**: `sk-NGY0NzM3NWEtNjljYy00Yzc0LWJlOTUtMTI3ZDJhMjEzN2Ni`
- **Tool ID** (for data insertion): `5d6a6efd-b231-45d2-a4fa-e05002c45c0a`
- **Tool Endpoint**: `https://api-bcbe5a.stack.tryrelevance.com/latest/studios/5d6a6efd-b231-45d2-a4fa-e05002c45c0a/trigger_webhook?project=248e7576-e3d3-4add-ac5c-d3072d3b4567`

7.3 N8N Instance

- **Status**: Self-hosted by client
- **Access**: Client has admin access
- **Workflows**: All workflows stored as JSON exports for backup

7.4 SAQ Portal (TO BE PROVIDED)

- **URL**: [To be confirmed by client]
- **Username**: [To be provided]
- **Password**: [To be provided]
- **API Credentials**: [To be investigated]

---

8. AI AGENT CONFIGURATION

8.1 Agent 1: Production & Supply Chain Intelligence Agent

Status: ✅ Prompt Created, Ready for Deployment

Purpose:

- Calculate raw material requirements for production

- Analyze ingredient consumption patterns
- Provide procurement intelligence
- Estimate production costs
- Track inventory by location

Primary Knowledge Bases Used:

1. `mrp_bom_line` (1,893 docs) - Critical for BOM analysis
2. `purchase_order_line_odoo` (320 docs) - Procurement data
3. `stock_quant_odoo` (200 docs) - Current inventory
4. `product_data_odoo` (200 docs) - Product master
5. `manufacturing_data_odoo` (190 docs) - BOM headers
6. `stock_movement_history` (305 docs) - Movement patterns
7. `purchase_order` (110 docs) - PO headers

Key Capabilities:

- Answer: "Can I produce 5,000 units of [product] right now?"
- Answer: "What raw materials am I missing?"
- Answer: "What's the material cost to produce [product]?"
- Answer: "Which suppliers do I buy from most?"
- Answer: "Show me consumption rate of [ingredient]"

Deployment Status: Ready to deploy once SAQ data is integrated (for enhanced forecasting)

8.2 Agent 2: Inventory Forecasting & Planning Agent

Status: ✅ Prompt Created, ⏳ Waiting for SAQ Data

Purpose:

- Generate demand forecasts based on sales data
- Identify stock-out risks
- Detect overstock situations
- Recommend production schedules

- Prioritize purchase orders

Primary Knowledge Bases Required:

1. `saq_sales_data` ⚠ **NOT YET AVAILABLE** - Critical for forecasting
2. `stock_movement_history` (305 docs) - Movement velocity
3. `stock_quant_odoo` (200 docs) - Current levels
4. `sale_order_line` (97 docs) - Limited internal sales
5. `product_data_odoo` (200 docs) - Product reference

Key Capabilities (Once SAQ data available):

- Answer: "What will be the demand for [product] next month?"
- Answer: "Which products are at risk of running out?"
- Answer: "Show me forecasted demand for Q1 2026"
- Answer: "What should I order from suppliers this week?"
- Calculate: Days of Inventory Remaining (DIR)
- Calculate: Reorder points and safety stock

Deployment Blocker: Requires SAQ sales data for accurate forecasting. Current Odoo sales data (97 lines) is insufficient for reliable predictions.

8.3 Future Agent Ideas

Agent 3: Customer & Geographic Intelligence (Post-SAQ Integration)

- Analyze sales by SAQ store location
- Identify best-performing regions
- Recommend targeted production based on geography
- Optimize distribution to high-demand areas

Agent 4: Financial Performance Agent

- Calculate profit margins by product
- Analyze supplier cost trends
- Recommend cost optimization strategies

- Track inventory carrying costs

Agent 5: Quality & Compliance Agent

- Track lot/batch information
- Monitor expiration dates
- Ensure FIFO/FEFO compliance
- Generate traceability reports

---

9. KEY TECHNICAL PATTERNS & BEST PRACTICES

9.1 Error Handling

- All Relevance API calls have `onError: continueRegularOutput`
- Failed records logged but don't stop workflow
- Retry logic built into batching

9.2 Data Quality Rules

- Never send `null` values to Relevance
- Never send placeholder values like `99999` or `"N/A"`
- Only include fields with valid data
- Use conditional JSON building

9.3 Monitoring & Validation

- Check KB record counts after each sync
- Compare expected vs actual records synced
- Validate critical fields (dates, IDs, quantities)
- Test with `limit: 10` before full sync

9.4 Scheduling Recommendations

- Odoo syncs: Every 15 minutes for real-time data

- SAQ syncs: Daily (frequency depends on SAQ capabilities)
- BOM syncs: Weekly (changes infrequently)
- Historical data: One-time backfill, then incremental updates

---

10. HANDOFF CHECKLIST

For Continuing SAQ Integration Work:

Client Tasks:

- [ ] Access SAQ B2B supplier portal
- [ ] Document available data export options
- [ ] Take screenshots of data structure
- [ ] Download sample export file (30-90 days)
- [ ] Check for API documentation
- [ ] Contact SAQ support about API availability
- [ ] Provide SAQ portal credentials to developer

Developer Tasks:

- [ ] Analyze SAQ data structure
- [ ] Design `saq_sales_data` KB schema
- [ ] Design `saq_product_mapping` KB schema
- [ ] Create product mapping template
- [ ] Build N8N workflow for SAQ data processing
- [ ] Implement data transformation logic
- [ ] Test with sample data
- [ ] Deploy to production
- [ ] Schedule regular syncs
- [ ] Deploy Forecasting Agent

Success Metrics:

- [ ] SAQ data flowing into Relevance AI
- [ ] >95% product matching accuracy
- [ ] At least 90 days historical data loaded
- [ ] Forecasting Agent operational
- [ ] Daily sync automation working

---

11. CONTACT & SUPPORT

Project Stakeholders:

- **Client**: Cherry River (Beverage Manufacturing)
- **Primary Contact**: [To be added]
- **Technical Lead**: [Developer handling implementation]

System Access:

- **Odoo Support**: [Odoo support contact]
- **Relevance AI Support**: support@relevanceai.com
- **N8N Community**: https://community.n8n.io

Documentation Links:

- **Odoo API Docs**: https://www.odoo.com/documentation/16.0/developer/reference/external_api.html
- **Relevance AI Docs**: https://docs.relevanceai.com
- **N8N Docs**: https://docs.n8n.io

---

12. APPENDICES

Appendix A: Complete N8N Workflow JSON

All workflows are stored as JSON exports. Key workflows:

1. `Odoo_Products_Sync.json`
2. `Odoo_Stock_Quant_Sync.json`
3. `Odoo_Purchase_Orders_Sync.json` (with nested lines)
4. `Odoo_Sale_Orders_Sync.json` (with nested lines)
5. `Odoo_BOM_Sync.json`
6. `Odoo_Stock_Movements_Sync.json`
7. `Odoo_MRP_Production_Sync.json`

[See attached document for full workflow JSON]

Appendix B: Sample Relevance AI Agent Prompts

Full prompts available in:

- Production_Supply_Chain_Agent_Prompt.md
- Inventory_Forecasting_Agent_Prompt.md

Appendix C: Data Mapping Tables

Odoo to Relevance Field Mappings: [See implementation docs]

SAQ to Odoo Product Mapping: [To be created after SAQ data analysis]

END OF DOCUMENT

This document should provide complete context for any developer to continue the SAQ integration work. All technical patterns, credentials, and implementation details are included.

Next Session Objective: Complete SAQ data discovery and implement first SAQ data sync to Relevance AI.